Set-Key Encoders for Discretization-Consistent Learning from Irregular Continuum Samples

Stepan Tretiakov
University of Texas at Austin
stepan@utexas.edu

Abstract

Set-valued observations often arise as irregular samples of an underlying continuum object. In practice, both measurement budgets and training computation are often limited. Under these constraints, an encoder should support training at a moderate sampling budget, improve systematically when denser observations become available at test time without retraining, and remain stable under changes in the sampling pattern. We introduce *Set-Key*, a set encoder architecture for irregular continuum samples. In Set-Key, coordinate-derived keys determine token–sensor interactions and value features carry the observed signal, while the additive pooling stage defines token-wise latent integral estimators. This identifies latent-space integration as the central architectural mechanism and exposes a clean insertion point for geometry-aware discretization control. We evaluate this idea under same-object resampling protocols that vary both sample count and sampling distribution while holding the underlying continuum signal fixed, using a simple k-nearest-neighbor density correction as one concrete realization of the geometry-aware pooling rule. Across a synthetic surface-average regression task, a real AhmedML surface-force benchmark, and a derived AirfRANS airfoil-force regression benchmark, Set-Key improves both matched-resolution robustness and zero-shot test-time refinement. In all three benchmarks, geometry-aware pooling reduces worst-case error under sampling shift, lowers prediction drift, and makes better use of additional test-time samples without retraining. These results support latent-space integration as a useful design principle for discretization-consistent set encoding from irregular continuum samples.

1 Introduction

Many machine learning problems receive set-valued observations that are, by construction, finite samples of an underlying continuum object. Examples include sampled surface fields in computational fluid dynamics, irregular boundary observations of aerodynamic profiles, and sensorized structural surfaces. In practice, training is often performed at a moderate sensor budget because both observation and computational costs increase with input resolution. At inference, however, the available number of sensors and the acquisition pattern may differ from those seen during training. A useful set encoder should therefore train efficiently on sparse inputs, exploit denser test-time observations without retraining, and remain reliable under shifts in the sampling strategy. Standard permutation-invariant encoders such as Deep Sets [4], Set Transformer [1], PointNet++ [2], and PointNeXt [3] aggregate over the empirical input set and are typically evaluated under matched train/test sampling, so this requirement is not built into their design.

The central difficulty is not only that the number of sensors may change, but that the empirical sampling measure may change with it. The same continuum object may be observed through uniform, biased, clustered, or partially occluded samples, and increasing the number of inference-time

sensors does not by itself remove this mismatch. If aggregation remains tied to the observation density, additional samples can reinforce the wrong latent summary rather than improve it. This issue is especially acute when the target depends on a continuum quantity, such as a surface-averaged signal or a global force coefficient, but it also affects any representation whose latent summaries should depend primarily on the underlying object rather than on the particular discretization. We refer to this requirement as *discretization consistency*.

This paper addresses discretization consistency through a particular encoder design. We introduce the *Set-Key* encoder, which decouples coordinate-derived keys from value-derived signal features for irregular continuum samples. In Set-Key, token-wise latent summaries are formed by an additive pooling stage, so each token can be interpreted as a latent-space integral estimator of a learned geometry-conditioned test function against the sampled signal. This makes the pooling stage the natural location for geometry-aware control of the sampling measure, while leaving the key, value, and readout pathways unchanged. Under this view, robustness to sampling-strategy shift is not an auxiliary property, but a direct consequence of how the encoder forms its latent summaries.

To assess discretization consistency directly, we introduce *same-object resampling protocols* that vary both sample count and sampling distribution while holding the underlying continuum signal fixed. The resulting evaluation targets the regime of practical interest for this paper: models are trained once at a moderate sensor budget and then tested under denser or differently distributed observations of the same object, without retraining. We report task accuracy together with prediction drift, worst-case performance across sampling modes, and refinement behaviour under increasing inference-time sample counts, so that modelling quality and sensitivity to the empirical sampling measure can be assessed separately.

We validate this perspective on three benchmarks spanning controlled and real-data settings: synthetic surface-average regression on $\mathbb{S}^2$, AhmedML sparse surface-force regression, and AirfRANS sparse airfoil-force regression. Across all three, Set-Key with geometry-aware pooling is more robust to sampling-strategy shift at matched resolution and benefits more systematically from additional inference-time samples without retraining. The largest gains appear precisely under biased and nonuniform sampling regimes, where standard additive aggregation remains most strongly tied to the empirical sampling measure.

2 Methods

2.1 Sampling Setup and Notation

We treat each input set as a finite observation of an underlying continuum surface or manifold signal. Let μ denote the normalized target measure over the object of interest, and let $(\boldsymbol{x}_i, \boldsymbol{u}_i)_{i=1}^M$ be sampled coordinates and values drawn from an observation process ν that may differ across views of the same underlying object. A model receives the sampled set $S = \{(\boldsymbol{x}_i, \boldsymbol{u}_i)\}_{i=1}^M$ and predicts a task output derived from the underlying continuum signal, such as a surface-average scalar or a global force coefficient. In the regime of interest for this paper, both the sample count M and the sampling pattern induced by ν may vary between training and inference.

Our goal is not only high task accuracy under i.i.d. train/test sampling, but stability under *same-object resampling*: if two observation sets come from the same underlying object but use different sample counts or density profiles, their encoded representations and predictions should remain close. This is the notion of discretization consistency studied throughout the paper.

2.2 Set-Key Encoder Architecture

The core model studied in this paper is a permutation-invariant set encoder that maps an unordered sensor set $\{(\boldsymbol{x}_i, \boldsymbol{u}_i)\}_{i=1}^M$, with $\boldsymbol{x}_i \in \mathbb{R}^{d_x}$ and $\boldsymbol{u}_i \in \mathbb{R}^{d_u}$, to p token-wise latent vectors $\{\boldsymbol{b}_k\}_{k=1}^p$, where $\boldsymbol{b}_k \in \mathbb{R}^{d_{\text{out}}}$. We refer to this structured encoder as a *Set-Key encoder*: token–sensor mixing is driven by a separate position-only key pathway, while sensor content is carried by a separate value pathway. The key point for this paper is that these latent tokens are formed by explicit additive aggregation, so they can be interpreted as learned latent-space integrals of geometry-conditioned test functions against sampled signals.

Input processing and sensor-side features. Let $\boldsymbol{e}_i$ denote the coordinate representation used by the encoder. In all experiments of this paper we take $\boldsymbol{e}_i = \boldsymbol{x}_i$ directly. A position-only Key Net $k_{\boldsymbol{\theta}} : \mathbb{R}^{d_e} \to \mathbb{R}^{d_k}$ maps the encoded sensor locations to key features

$$\boldsymbol{K}_i = k_{\boldsymbol{\theta}}(\boldsymbol{e}_i) \in \mathbb{R}^{d_k},$$

while a Value Net produces sensor-side value features

$$\boldsymbol{V}_i = v_{\boldsymbol{\theta}}(\boldsymbol{u}_i) \in \mathbb{R}^{d_v} \qquad \text{or} \qquad \boldsymbol{V}_i = v_{\boldsymbol{\theta}}([\boldsymbol{e}_i, \boldsymbol{u}_i]) \in \mathbb{R}^{d_v}.$$

Both variants fit the present formulation. In the experiments of this paper we use the latter option, so the value pathway may depend on both geometry and signal, while token–sensor mixing itself remains controlled solely by the coordinate-derived keys.

Token–sensor aggregation. Let $\mathbf{Q} \in \mathbb{R}^{p \times d_k}$ denote a matrix of learnable query tokens, and let $\mathbf{K} = [\boldsymbol{K}_1, \ldots, \boldsymbol{K}_M]^\top \in \mathbb{R}^{M \times d_k}$ and $\mathbf{V} = [\boldsymbol{V}_1, \ldots, \boldsymbol{V}_M]^\top \in \mathbb{R}^{M \times d_v}$ denote sensor-side key and value features, respectively. The token–sensor score matrix is

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \in \mathbb{R}^{p \times M}.$$

The implementation optionally applies a learned temperature and then a pointwise basis nonlinearity $a(\cdot)$ to $\mathbf{S}$, with $a \in \{\tanh, \text{softsign}, \text{softplus}\}$. In all experiments we use the softplus choice and write

$$\mathbf{B} = a(\mathbf{S}) \in \mathbb{R}^{p \times M}.$$

Let $\boldsymbol{w} \in \mathbb{R}_+^M$ denote sensor weights and $\boldsymbol{m} \in \{0,1\}^M$ an optional validity mask. Under the "total" normalization used throughout the experiments, the token matrix is

$$\mathbf{P} = \frac{\mathbf{B}\ \text{diag}(\boldsymbol{m} \odot \boldsymbol{w})\ \mathbf{V}}{\sum_{i=1}^M m_i w_i + \varepsilon} \in \mathbb{R}^{p \times d_v}. \tag{1}$$

Equivalently, for the k-th token,

$$\boldsymbol{P}_k \;=\; \sum_{i=1}^M \tilde{w}_i\, a(S_{k,i})\, \boldsymbol{V}_i, \qquad k = 1, \ldots, p.$$

where

$$\tilde{w}_i \;=\; \frac{m_i w_i}{\sum_{j=1}^M m_j w_j + \varepsilon}, \qquad i = 1, \ldots, M.$$

The scalar weights $\boldsymbol{w}$ encode the geometry-aware correction of the empirical sampling measure. Uniform pooling is recovered by taking $w_i \equiv 1$, while nonuniform choices of $\boldsymbol{w}$ alter only the final additive pooling rule and are shared across all tokens.

Readout to token coefficients. The pooled token summaries are post-processed row-wise by a small map $\rho_{\text{tok}} : \mathbb{R}^{d_v} \to \mathbb{R}^{d_{\text{out}}}$, yielding

$$\boldsymbol{b}_k = \rho_{\text{tok}}(\boldsymbol{P}_k), \qquad k = 1, \ldots, p.$$

In the benchmarks considered here, the p output tokens are flattened and passed through a task head.

2.3 Additive Pooling as a Latent-Space Integration Operator

Equation (1) identifies the additive pooling stage as the site where the latent-integration interpretation becomes explicit. Writing the predictor as

$$\widehat{\boldsymbol{y}}(S) \;=\; r\big(\boldsymbol{P}_1(S), \ldots, \boldsymbol{P}_p(S)\big),$$

where $S = \{(\boldsymbol{x}_i, \boldsymbol{u}_i)\}_{i=1}^{M}$ is the observed set, $\boldsymbol{P}_k(S)$ are the pooled token summaries from Equation (1), and r denotes the downstream readout (combining the row-wise map ρ_{tok} with the final task head), the quadrature-style interpretation applies to the intermediate token summaries $\boldsymbol{P}_k$, and the readout acts on the resulting latent summaries.

For a fixed token k, the pooling rule has the form

$$\boldsymbol{P}_k(S) \;=\; \sum_{i=1}^{M} \tilde{w}_i \, \phi_k(\boldsymbol{x}_i) \, \psi(\boldsymbol{x}_i, \boldsymbol{u}_i), \tag{2}$$

where ϕ_k is a learned scalar test function indexed by token k, ψ is a vector-valued value map, and $\tilde{w}_i$ are normalized aggregation weights. When the sampled set approximates a continuum signal $\boldsymbol{u} : \mathcal{M} \to \mathbb{R}^{d_u}$ over a target measure μ, this expression is a quadrature-style estimator of the latent functional

$$\bar{\boldsymbol{P}}_k \;=\; \int_{\mathcal{M}} \phi_k(\boldsymbol{x}) \, \psi(\boldsymbol{x}, \boldsymbol{u}(\boldsymbol{x})) \, \mathrm{d}\mu(\boldsymbol{x}). \tag{3}$$

Accordingly, the readout r operates on a finite collection of latent integral estimators rather than on raw samples.

This decomposition yields an immediate stability relation. If the readout $r : \mathbb{R}^{pd_v} \to \mathbb{R}^{d_{\text{out}}}$ is L_r-Lipschitz under the Euclidean norm, and if

$$\mathbf{P}(S) = \operatorname{vec}\big[\boldsymbol{P}_1(S), \ldots, \boldsymbol{P}_p(S)\big] \in \mathbb{R}^{pd_v},$$

then token-level integration error controls output error:

$$\|\widehat{\boldsymbol{y}}(S) - \widehat{\boldsymbol{y}}(S')\| \;\le\; L_r \, \big\|\mathbf{P}(S) - \mathbf{P}(S')\big\|.$$

Hence, improvements in the additive pooling rule improve the quality of the latent integral estimators supplied to the readout, and the induced stability gain propagates through the final predictor.

Uniform pooling corresponds to the special case $\tilde{w}_i = 1/M$, which is correct only when the observation process already matches the target measure μ. If the same object is observed under a shifted sampling density $\nu \neq \mu$, uniform pooling converges to the wrong latent functional even though permutation invariance is preserved. This token-level mismatch is the basic source of discretization-induced prediction drift.

2.4 Geometry-Aware Weighting via kNN Density Estimates

As a simple geometry-aware baseline, we estimate local sampling density from k-nearest-neighbor radii and use those estimates to reweight the additive pooling rule. If $r_k(\boldsymbol{x}_i)$ denotes the distance from point $\boldsymbol{x}_i$ to its k-th nearest neighbor, then for an intrinsic dimension d we define

$$w_i \;\propto\; r_k(\boldsymbol{x}_i)^d, \tag{4}$$

followed by normalization so that $\sum_i w_i = 1$ over the observed set. Intuitively, densely sampled regions have smaller r_k and therefore receive less mass, while sparse regions receive more. This makes the rule a crude local cell-area estimator and therefore a natural baseline when the target measure differs from the observation density.

We do not regard this k-NN rule as a final answer. It is a useful heuristic density correction and an effective empirical baseline, but it captures only one scalar notion of local scale and does not explicitly model anisotropy, curvature, or missing support. One goal of the benchmark suite is to quantify exactly how far such simple geometry-aware weighting goes and where stronger aggregation rules will be needed.

2.5 Discretization Consistency

We call an encoder discretization consistent if its output depends primarily on the underlying continuum object and only weakly on the specific finite discretization used to observe that object. Formally, if two sets S, S' are sampled from the same continuum object under different resolutions or densities, then a discretization-consistent encoder should satisfy $E(S) \approx E(S')$ and therefore produce stable downstream predictions.

This requirement is distinct from ordinary generalization. A model can achieve high i.i.d. accuracy while still reacting strongly to harmless changes in sampling density. Our empirical metrics therefore separate task quality from representation stability: prediction drift measures output sensitivity under same-object resampling, and worst-case performance across sampling modes measures robustness.

For a test object o, let S_o^{ref} denote a dense uniform reference view and let $S_{o,m,N,r}$ denote a resampled view under sampling mode m, point count N, and replica r. Writing $\widehat{\boldsymbol{y}}_o^{\mathrm{ref}} = \widehat{\boldsymbol{y}}(S_o^{\mathrm{ref}})$ and $\widehat{\boldsymbol{y}}_{o,m,N,r} = \widehat{\boldsymbol{y}}(S_{o,m,N,r})$, we define the prediction drift for that setting as

$$D_{m,N} \;=\; \frac{1}{|T|\,R\,d_{\mathrm{out}}} \sum_{o\in T} \sum_{r=1}^{R} \big\|\widehat{\boldsymbol{y}}_{o,m,N,r} - \widehat{\boldsymbol{y}}_o^{\mathrm{ref}}\big\|_1,$$

where T is the test set, R is the number of resampling replicas, and d_{out} is the output dimension. Thus, the table entry labelled "Pred. Drift" is the mean absolute change in prediction relative to the dense reference view of the same object.

3 Experiments

The experiments evaluate Set-Key in the regime that motivates this paper: training at a moderate sensor budget, followed by inference under denser or differently distributed observations of the same underlying object without retraining. The benchmark suite covers three continuum settings: a controlled synthetic surface-functional regression task on $\mathbb{S}^2$, a real AhmedML surface-force regression benchmark, and a derived AirfRANS airfoil-force regression benchmark. Within the Set-Key family we compare two matched models that differ only in the pooling weights: a **uniform** encoder with

	Synthetic Regr.			AhmedML			AirfRANS		
Model	In-dist. RMSE	Worst RMSE	Pred. Drift	In-dist. RMSE	Worst RMSE	Pred. Drift	In-dist. RMSE	Worst RMSE	Pred. Drift
Uniform	0.046	0.248	0.159	0.023	0.065	0.016	0.022	0.100	0.036
kNN	**0.021**	0.188	**0.087**	**0.022**	0.037	**0.008**	**0.017**	**0.056**	**0.013**
PointNeXt	0.042	**0.146**	0.171	0.024	**0.037**	0.031	0.024	0.063	0.219

Table 1: Main matched-resolution results. Each task is evaluated at its training resolution, so this table isolates density-shift robustness from resolution shift. We report in-distribution RMSE, worst-case RMSE, and average nonuniform prediction drift for both the synthetic surface-functional task together with the two real-data benchmarks: AhmedML sparse surface-force regression and AirfRANS sparse airfoil-force regression. Here "In-dist. RMSE" denotes RMSE under uniform sampling at the training resolution, "Worst RMSE" denotes the largest RMSE across sampling modes at that same resolution, and "Pred. Drift" denotes the mean absolute difference between a prediction from a sparse or shifted view and the dense-reference prediction for the same object, averaged over target components and over nonuniform sampling modes.

standard additive pooling and a **k-NN density** encoder with geometry-aware weighting (k=8). All models are trained on uniformly sampled observations and then evaluated under same-object resampling protocols that vary both the sampling pattern and the number of observed samples. We report task quality together with *discretization-consistency* metrics—prediction agreement with a high-resolution reference, prediction drift, and worst-case performance across sampling modes—so that modelling quality and stability under sampling shift are assessed separately.

For architecture-level context we also report results with a PointNeXt-style [3] backbone as a strong reference model with learned local aggregation. This comparison tests whether the gains from geometry-aware pooling in Set-Key remain visible relative to a competitive modern point-set architecture, rather than only relative to uniform variants of the same encoder. Because PointNeXt aggregates features through learned local neighborhood blocks and does not expose an explicit scalar-weight insertion point analogous to the $\boldsymbol{w}$ in Equation (1), we treat it as a reference architecture rather than as a geometry-weighted variant of Set-Key.

Quantitative summaries. Tables 1, 2, 3, 4, and 5 provide the main quantitative summaries. Table 1 gives the cross-benchmark comparison for Set-Key across synthetic surface-functional regression together with the AhmedML and AirfRANS real-data force-regression benchmarks. Tables 2 and 3 summarize the matched-resolution sampling breakdown and refinement behaviour on AhmedML, while Tables 4 and 5 provide the corresponding summaries for AirfRANS.

3.1 Surface-Functional Regression under Resampling

Our first controlled benchmark targets a continuum functional with a known measure: the surface average of a scalar field on $\mathbb{S}^2$. This setting isolates the role of aggregation weights particularly clearly, since the target depends directly on integration over the underlying surface rather than on a downstream semantic label.

Setup. Each object is a continuous scalar field on the unit sphere $\mathbb{S}^2$ of the form

$$f(\mathbf{p}) \;=\; b + \mathbf{p}^\top \boldsymbol{\ell} + \boldsymbol{\phi}(\mathbf{p})^\top \mathbf{q} + \sum_{j=1}^{4} a_j \exp\big[\kappa_j \big(\mathbf{p}^\top \mathbf{c}_j - 1\big)\big], \tag{5}$$

where b is a scalar bias, $\boldsymbol{\ell} \in \mathbb{R}^3$ and $\mathbf{q} \in \mathbb{R}^5$ are linear and quadratic coefficients over hand-crafted polynomial features $\boldsymbol{\phi}$, and the final sum adds four localized Gaussian-like bumps with random centers $\mathbf{c}_j \in \mathbb{S}^2$, amplitudes a_j, and concentrations $\kappa_j \in [8, 22]$. All parameters are drawn independently for each object. We generate 2 048 training, 256 validation, and 512 test objects. The target quantity is the surface average $\bar{f} = \int_{\mathbb{S}^2} f \, \mathrm{d}\sigma$, approximated using 4 096 uniform reference points.

Both regressors share the same weighted set encoder architecture: 16 learnable query tokens, key dimension 64, token dimension 32, hidden dimension 128, and a three-layer MLP head. They differ only in their aggregation weights—$1/N$ or k-NN density estimates (k=8, intrinsic dimension d=2). Each model is trained for 2 000 steps on batches of 64 objects, each observed through 128 uniformly sampled points, using AdamW (learning rate 10^{-3}, weight decay 10^{-5}) with gradient clipping at norm 1.

Evaluation protocol. At test time we resample each object under five density distributions—*uniform*, *polar* (concentrated near the poles), *equatorial* (concentrated near the equator), *clustered* (concentrated around two random centers), and *hemisphere* (visible from a random viewpoint)—at each of five point counts $N \in \{32, 64, 128, 256, 512\}$, with three independent resampling replicas per setting. For every object we also obtain a *reference prediction* from a dense 2 048-point uniform sample. This reference is not ground truth; it is the model's own best-informed output, and serves as an anchor for measuring how much the prediction shifts when the input is degraded or redistributed.

Metrics. We report four quantities per setting: *RMSE* (root mean squared error against the Monte Carlo estimate of the surface average), *MAE* (mean absolute error), *bias* (signed mean error, indicating systematic over- or under-prediction), and *prediction drift* (mean absolute difference from the reference prediction). The first three characterize task-level accuracy; the last isolates discretization instability. A model can achieve low bias yet high drift—it is correct on average but gives a different answer each time the same object is resampled. This error–drift decomposition is the central diagnostic: it separates estimation quality from representation stability in a way that scalar accuracy alone cannot. We summarize with worst-case RMSE across modes and average non-uniform prediction drift.

Observed pattern. This synthetic regression task is the controlled case where the quadrature interpretation is most directly aligned with the target: the model is asked to recover a surface average from irregular samples. Table 1 therefore gives a matched-resolution summary. At the training resolution, k-NN weighting lowers in-distribution RMSE from 0.046 to 0.021, lowers worst-case RMSE under sampling shift from 0.248 to 0.188, and reduces average nonuniform prediction drift from 0.159 to 0.087. The same trend is visible under refinement in Figure 1: the geometry-aware encoder follows a consistently lower error curve and exhibits a steeper reduction in same-object prediction drift as the number of observed points grows, while PointNeXt serves as a larger-backbone reference rather than an explicitly weighted quadrature model. Figure 2 shows the same comparison at the prediction level for one challenging density shift: under polar sampling at N=512,

the k-NN encoder produces a noticeably tighter true-versus-predicted scatter and a smaller residual cloud than uniform pooling, while the PointNeXt reference remains competitive but more dispersed on this synthetic functional estimation task.

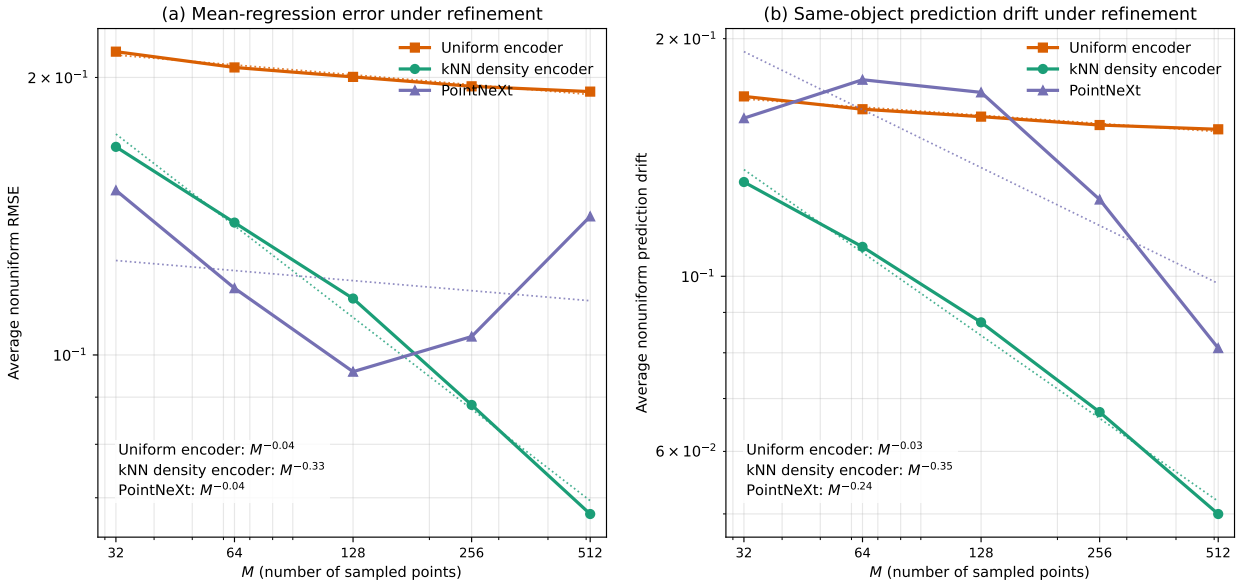

Figure 1: Synthetic mean-regression refinement curves. Average nonuniform RMSE (left), shown here as a smoother refinement diagnostic complementary to the worst-case table metric, and average nonuniform prediction drift (right), defined as the mean absolute change from the dense-reference prediction of the same object, as the number of observed sphere samples increases. Dotted lines show log-log power-law fits.

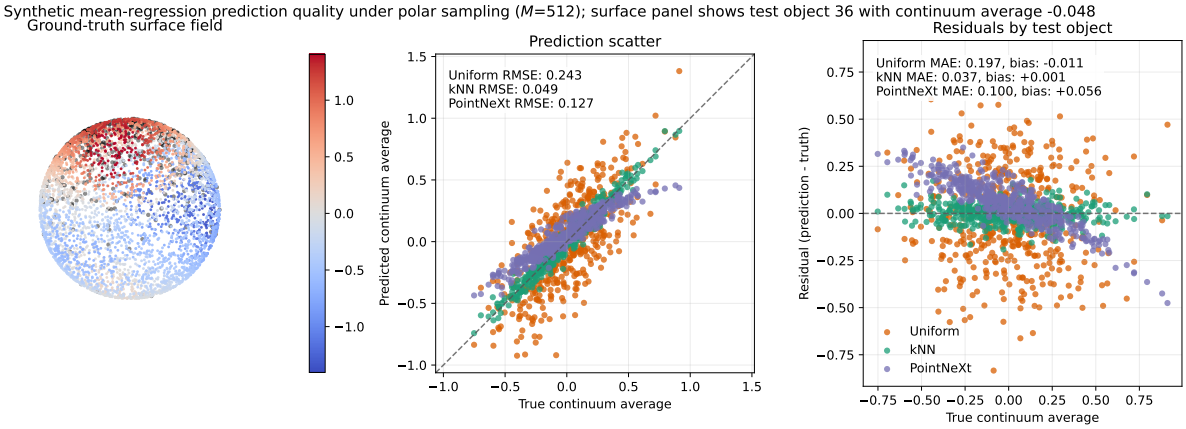

Figure 2: Synthetic mean-regression predictions under polar sampling at N=512. A representative test sphere colored by the ground-truth scalar field with observed samples overlaid (left), true-versus-predicted scatter for the Monte Carlo surface-average target (middle), and residuals relative to that target (right) for uniform pooling, k-NN weighting, and PointNeXt.

Model	uniform	front	rear	roof	clustered	occluded	Avg. Nonunif.	Worst
Uniform	0.023	0.065	0.029	0.028	0.025	0.023	0.034	0.065
kNN	**0.022**	0.037	**0.021**	**0.023**	**0.023**	**0.021**	**0.025**	0.037
PointNeXt	0.024	**0.036**	0.026	0.037	0.025	0.025	0.030	**0.037**

Table 2: Sampling-mode breakdown on the AhmedML benchmark at the matched training resolution $N = 256$. Entries report RMSE under each sampling mode, followed by the average over nonuniform modes and the worst-case value across all modes.

	Worst RMSE ↓					Pred. Drift ↓				
Model	64	128	256	512	1024	64	128	256	512	1024
Uniform	0.080	0.069	0.065	0.053	0.054	0.023	0.018	0.016	0.014	0.013
kNN	**0.075**	**0.045**	0.037	**0.023**	**0.024**	**0.021**	**0.013**	**0.008**	**0.005**	**0.003**
PointNeXt	0.119	0.069	**0.037**	0.033	0.061	0.052	0.032	0.031	0.027	0.017

Table 3: Resolution/refinement study on the AhmedML benchmark. The left block reports the primary worst-case task metric across point counts, while the right block reports the corresponding same-object consistency metric, namely average nonuniform prediction drift relative to the dense-reference prediction for the same object. This table is intended to expose whether gains from geometry-aware weighting persist as the discretization is refined.

3.2 AhmedML Surface-Force Regression

Task. To test whether the same aggregation mechanism transfers beyond the synthetic sphere setting, we consider a real sparse surface-functional benchmark derived from the AhmedML dataset of Ahmed-body CFD simulations. Each example consists of a triangulated vehicle surface together with the corresponding force coefficients. We preprocess each mesh into an irregular set of cell-centered surface samples with associated area weights, using cell centers as coordinates and concatenating local pressure, cell normals, and wall-shear vectors as the observed per-cell signal. The targets are the drag and lift coefficients (C_d, C_l), so the task is multivariate regression of physically meaningful global surface functionals from sparse irregular surface observations.

Setup and resampling protocol. The benchmark uses all 500 processed AhmedML runs, split into 350 training, 75 validation, and 75 test bodies. Each model is trained on 256 uniformly sampled surface cells per object and evaluated under six sampling modes: *uniform*, *front*, *rear*, *roof*, *clustered*, and *occluded*. The nonuniform modes are obtained by reweighting the base surface-area distribution, so the sampling pattern changes while the underlying CFD solution remains fixed. We evaluate at point counts $N \in \{64, 128, 256, 512, 1024\}$ with three independent resampling replicas per setting, and use a dense 2 048-cell reference view to measure same-object prediction drift.

Metrics. We summarize AhmedML with the same measure-sensitive diagnostics used in the synthetic regression task: in-distribution RMSE at the matched training resolution, worst-case RMSE across sampling modes, and average nonuniform prediction drift from the dense reference prediction. Table 1 reports the cross-task summary, Table 2 gives the per-mode breakdown at the matched training resolution N=256, and Table 3 shows how worst-case error and drift change as the sampling budget grows.

Observed pattern. AhmedML is the primary real-data case in the paper where geometry-aware pooling helps on the task metric itself rather than only on robustness proxies. At the matched

training resolution N=256, k-NN weighting reduces worst-case RMSE from approximately 0.065 to 0.037 and average nonuniform prediction drift from approximately 0.016 to 0.008, with the largest gains appearing under the front and clustered sampling modes. Figure 3 shows the same effect as a refinement plot: the geometry-aware encoder follows a steeper decay curve for both worst-case error and same-object prediction drift as the surface-cell budget grows, even against the PointNeXt reference architecture.

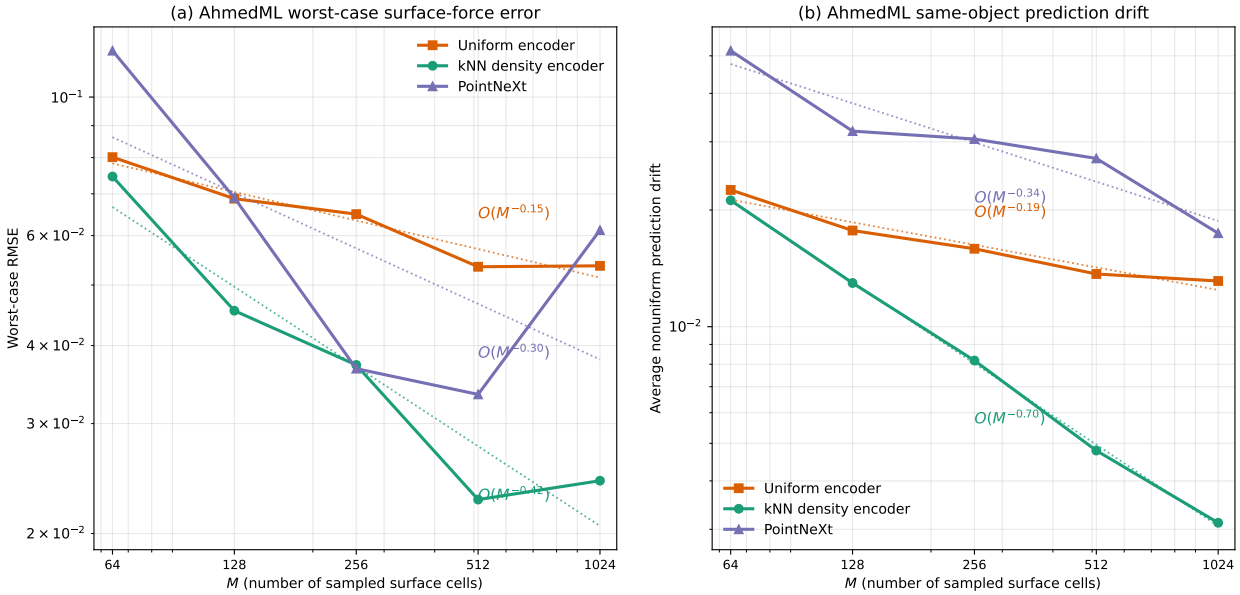

Figure 3: AhmedML refinement curves. Worst-case RMSE across sampling modes (left) and average nonuniform prediction drift (right), defined as the mean absolute change from the dense-reference prediction of the same object, as the surface-cell budget grows. Dotted lines show log-log power-law fits.

To complement the aggregate tables, we also include one qualitative prediction figure for AhmedML. Figure 4 combines a representative front-biased surface view with test-set scatter plots for C_d and C_l under the same sampling condition at the matched training resolution N=256. The figure is shown at a denser inference condition N=1024 to illustrate the same-object prediction quality under refinement. This makes the benchmark visually concrete while also showing the actual regression quality of the two Set-Key variants together with a PointNeXt reference.

3.3 AirfRANS Airfoil-Force Regression

Task. To complement AhmedML with a second real aerodynamic functional benchmark, we derive a sparse airfoil-force regression task from the raw AirfRANS CFD simulations. Each example is an irregular sample of the airfoil boundary curve. We preprocess every simulation into an airfoil point set with coordinates, pressure, wall-shear stress, and boundary normals, and take the global targets to be the aerodynamic force coefficients (C_d, C_l) computed with the AirfRANS reference API.

Setup and resampling protocol. We use the official AirfRANS "full" split as prepared by the benchmark preprocessing pipeline, yielding 720 training, 80 validation, and 200 test simulations. Each model is trained for 25 000 steps on 256 uniformly sampled airfoil boundary points and then

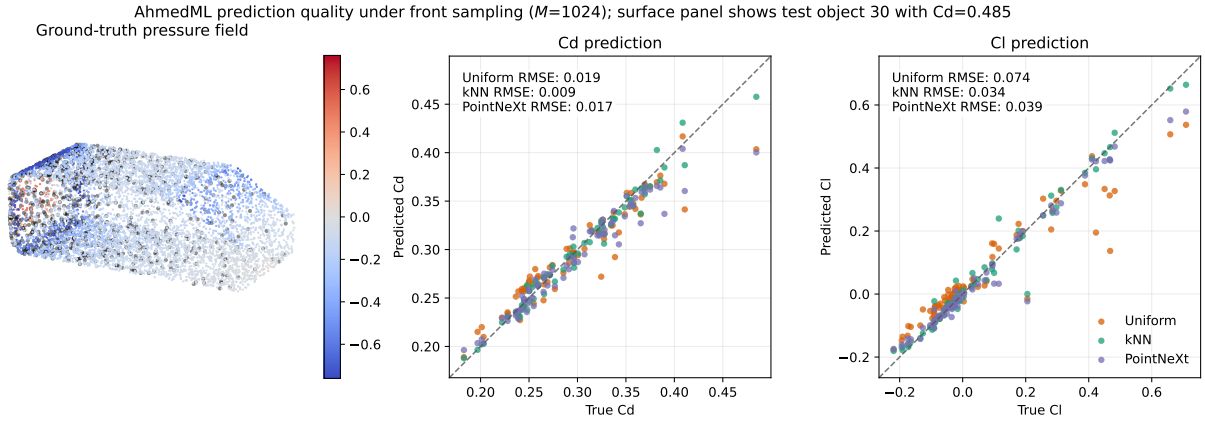

Figure 4: AhmedML predictions under front-biased sampling at N=1024. A representative test body with observed cells overlaid (left), and true-vs-predicted scatter for C_d (middle) and C_l (right) comparing the uniform Set-Key variant, the geometry-aware k-NN Set-Key variant, and PointNeXt.

Model	uniform	leading edge	trailing edge	upper surface	lower surface	clustered	occluded	Avg. Nonunif.	Worst
Uniform	0.022	0.081	0.100	0.078	0.056	0.042	0.024	0.063	0.100
kNN	**0.017**	**0.028**	**0.029**	0.056	0.043	**0.022**	**0.021**	**0.033**	**0.056**
PointNeXt	0.024	0.058	0.063	**0.028**	**0.032**	0.031	0.027	0.040	0.063

Table 4: Sampling-mode breakdown on the AirfRANS benchmark at the matched training resolution $N = 256$. Entries report RMSE under each sampling mode, followed by the average over nonuniform modes and the worst-case value across all modes.

evaluated under seven observation modes: *uniform*, *leading edge*, *trailing edge*, *upper surface*, *lower surface*, *clustered*, and *occluded*. These modes reweight the base boundary-length distribution, so the observed subset shifts around the same underlying airfoil solution while the reference coefficients remain fixed. We test at point counts $N \in \{64, 128, 256, 512, 1024\}$ and use a dense 2 048-point reference view for same-simulation prediction drift.

Metrics. For AirfRANS we report the same diagnostics used for AhmedML: matched-resolution RMSE on (C_d, C_l) at N=256, worst-case RMSE across sampling modes, and average nonuniform prediction drift from the dense reference prediction on the same simulation. Table 1 includes the matched-resolution summary; Table 4 and Table 5 give the detailed sampling-mode and refinement breakdowns.

Observed pattern. This derived AirfRANS force-regression task supports the same matched-resolution conclusion as AhmedML, while showing a more heterogeneous refinement pattern at very low sampling budgets. At the matched training resolution N=256, k-NN weighting lowers in-distribution RMSE from approximately 0.0218 to 0.0168, lowers worst-case RMSE from approximately 0.0998 to 0.0565, and reduces average nonuniform prediction drift from approximately 0.0359 to 0.0130. The PointNeXt reference is competitive on worst-case RMSE at the matched resolution, but it remains much less discretization-stable, with drift above 0.21. Under refinement, the k-NN encoder reaches worst-case RMSE below 0.019 and prediction drift below 0.006 by N=1024, while both uniform pooling and PointNeXt deteriorate under the most adverse nonuniform views.

	Worst RMSE ↓					Pred. Drift ↓				
Model	64	128	256	512	1024	64	128	256	512	1024
Uniform	**0.146**	**0.120**	0.100	0.077	0.157	**0.050**	0.044	0.036	0.023	0.064
kNN	0.193	0.134	**0.056**	**0.018**	**0.019**	0.054	**0.030**	**0.013**	**0.005**	**0.006**
PointNeXt	0.198	0.136	0.063	0.129	0.349	0.206	0.205	0.219	0.192	0.094

Table 5: Resolution/refinement study on the AirfRANS benchmark. The left block reports the primary worst-case task metric across point counts, while the right block reports the corresponding same-object consistency metric, namely average nonuniform prediction drift relative to the dense-reference prediction for the same object. This table is intended to expose whether gains from geometry-aware weighting persist as the discretization is refined.

Figure 5 shows the same empirical refinement trend directly. Because the reported quantity is the worst-case error across several resampling modes at each point count, monotonic decay is not expected for every model; the relevant feature is the separation between methods as the sampling budget increases. In that comparison, the geometry-aware encoder separates from both uniform pooling and PointNeXt in both worst-case error and same-object drift. Figure 6 shows the same comparison at the per-simulation level under a trailing-edge sampling pattern: the left panel visualizes a representative test airfoil with the observed boundary samples overlaid, while the scatter plots on the right show that k-NN weighting stays much closer to the true (C_d, C_l) targets than either uniform pooling or the PointNeXt reference.

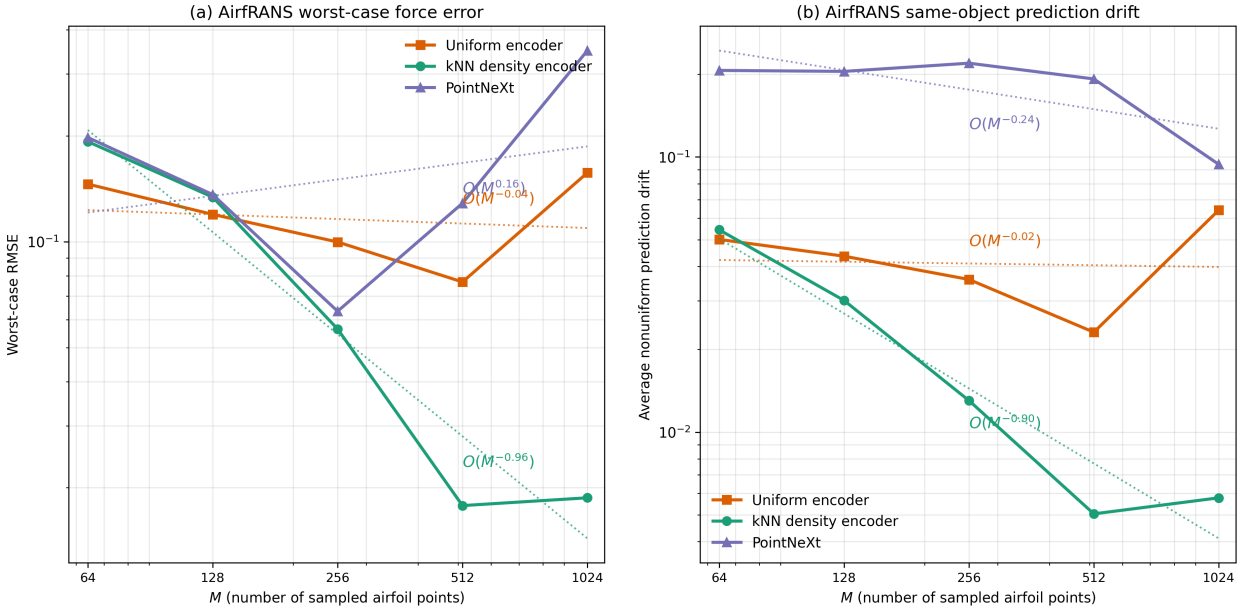

Figure 5: AirfRANS refinement curves. Worst-case RMSE across sampling modes (left) and average nonuniform prediction drift (right), defined as the mean absolute change from the dense-reference prediction of the same simulation, as the sampled airfoil-point budget grows. Dotted lines show log-log power-law fits.

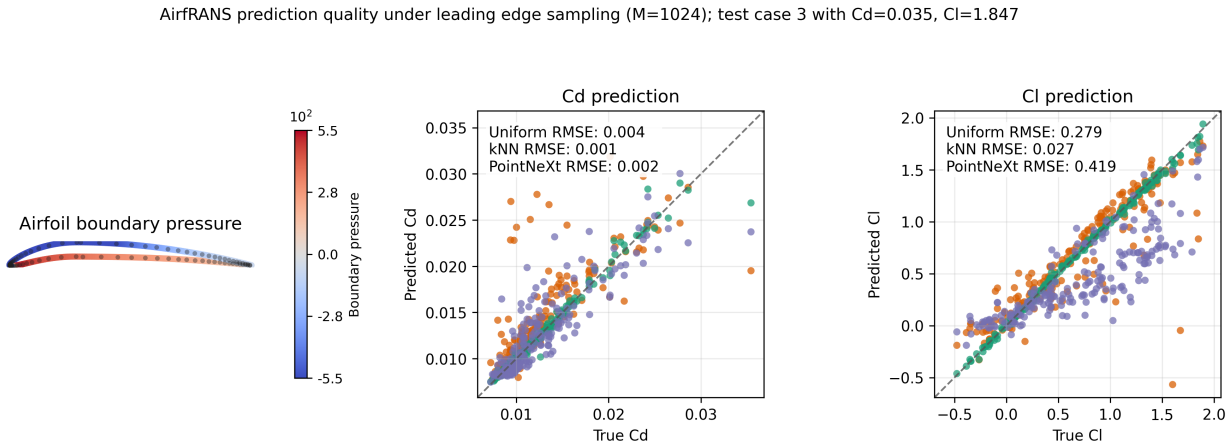

Figure 6: AirfRANS predictions under trailing-edge-biased sampling at N=1024. Left: a representative test airfoil colored by boundary pressure, with the observed boundary samples overlaid. Middle and right: true-versus-predicted scatter plots for C_d and C_l across the full AirfRANS test split, comparing uniform pooling, the geometry-aware k-NN encoder, and PointNeXt under the same observation pattern. The diagonal line indicates perfect prediction.

References

[1] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant input. In *International Conference on Machine Learning*, pages 3744–3753, 2019.

[2] Charles R Qi, Li Yi, Hao Su, and Leonidas J Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[3] Guocheng Qian, Yuchen Li, Houwen Peng, Jinjie Mai, Hasan Hammoud, Mohamed Elhoseiny, and Bernard Ghanem. PointNeXt: Revisiting PointNet++ with improved training and scaling strategies. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[4] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabás Póczos, Ruslan R Salakhutdinov, and Alexander J Smola. Deep sets. In *Advances in Neural Information Processing Systems*, volume 30, 2017.